

University Of Moratuwa
Department of Electronic & Telecommunication Engineering

EN3150 Pattern Recognition

Assignment 03

Simple convolutional neural network to perform classification.

Samarasekara S.M.R.P. - 230564L
Kaushalya R.G.S.P. - 230327N
Jayakody J.A.C.P. - 230268H
Jayaweera N.S. - 230300C

1 Custom Architecture Design

1.1 Model A: Standard CNN

Table 1: Architecture summary of Model_A_Standard_CNN

Layer (type)	**Output Shape**	**Param #**
conv2d_3 (Conv2D)	(None, 64, 64, 32)	896
max_pooling2d_6 (MaxPooling2D)	(None, 32, 32, 32)	0
conv2d_4 (Conv2D)	(None, 32, 32, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 16, 16, 64)	0
conv2d_5 (Conv2D)	(None, 16, 16, 128)	73,856
max_pooling2d_8 (MaxPooling2D)	(None, 8, 8, 128)	0
flatten_1 (Flatten)	(None, 8192)	0
dense_4 (Dense)	(None, 128)	1,048,704
dense_5 (Dense)	(None, 6)	774
Total params		1,142,726 (4.36 MB)
Trainable params		1,142,726 (4.36 MB)
Non-trainable params		0

1.2 Model B: Lightweight CNN

Table 2: Architecture summary of Model_B_Lightweight_CNN

Layer (type)	**Output Shape**	**Param #**
separable_conv2d_3 (SeparableConv2D)	(None, 64, 64, 32)	155
max_pooling2d_9 (MaxPooling2D)	(None, 32, 32, 32)	0
separable_conv2d_4 (SeparableConv2D)	(None, 32, 32, 64)	2,400
max_pooling2d_10 (MaxPooling2D)	(None, 16, 16, 64)	0
separable_conv2d_5 (SeparableConv2D)	(None, 16, 16, 128)	8,896
max_pooling2d_11 (MaxPooling2D)	(None, 8, 8, 128)	0
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 128)	0
dense_6 (Dense)	(None, 64)	8,256
dense_7 (Dense)	(None, 6)	390
Total params		20,097 (78.50 KB)
Trainable params		20,097 (78.50 KB)
Non-trainable params		0

1.3 Hardware-Aware Selection of Activation Functions

ReLU is computationally efficient because it requires only a comparison with zero, unlike sigmoid and tanh, which involve computationally expensive exponential operations. This makes it suitable for resource-constrained embedded processors and edge devices. ReLU also helps mitigate the vanishing gradient problem for positive inputs.

Softmax was selected for the output layer because the task involves six mutually exclusive scene categories. It converts logits into normalized class probabilities:

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{6} e^{z_j}},$$

where z_i is the logit for class i. Although Softmax involves exponential calculations, its computational overhead is limited because it operates on only six output values per image.

Both models use the same activation functions for a consistent comparison. ReLU supports efficient feature learning, while Softmax provides class probabilities for multiclass prediction. Actual inference speed and energy consumption also depend on the hardware and implementation.

1.4 Optimizer Selection and Tuning

1.4.1 Experimental Setup

Model A was trained for 15 epochs using Adam, standard Stochastic Gradient Descent (SGD), and SGD with momentum. The learning rates were set to 10^{-3} for Adam and 10^{-2} for both SGD configurations, with a momentum coefficient of 0.9 for SGD with momentum. Validation accuracy and loss were used to compare their performance.

1.4.2 Comparison of Optimizers

Figure 1 compares the training loss, validation loss, and validation accuracy of the three optimizers.



Figure 1: Comparison of Adam, SGD, and SGD with momentum for Model A.

Adam achieved the highest best validation accuracy of 82.04%, followed by SGD with momentum at 79.69% and standard SGD at 71.78%. Adam also achieved faster initial convergence, making it the most effective optimizer under the tested settings. Its adaptive learning rates help adjust parameter updates according to gradient statistics.

1.4.3 Effect of Momentum

To investigate momentum, SGD was evaluated with momentum values of 0.0, 0.5, 0.9, and 0.99, while keeping the learning rate at 10^{-2}. Figure 2 shows the resulting training loss, validation loss, and validation accuracy.

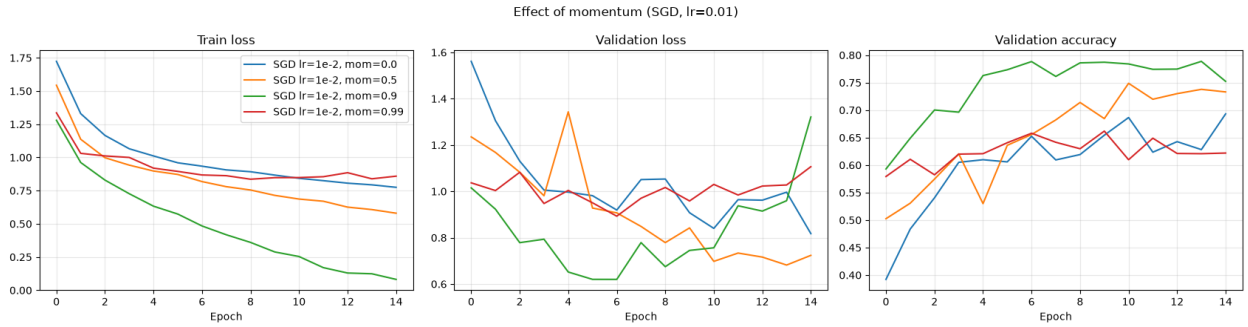


Figure 2: Effect of momentum on SGD performance.

The best validation accuracy improved from 69.32% with no momentum to 74.87% and 78.86% with momentum values of 0.5 and 0.9, respectively. However, it decreased to 66.18% at 0.99. Momentum accumulates information from previous updates, helping accelerate progress in consistent gradient directions. However, excessive momentum can lead to overshooting and reduced performance.

1.4.4 Selected Optimizer

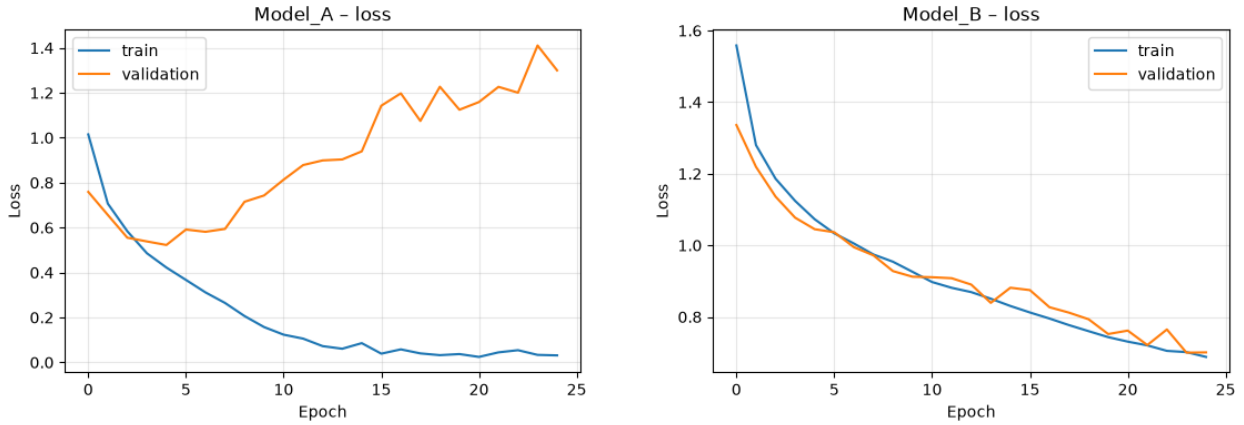
Adam with a learning rate of 10^{-3} was selected for the final training of both CNN models because it achieved the highest validation accuracy among the main optimizer configurations. Both models were subsequently trained for 25 epochs using the same optimizer settings for a consistent comparison.

2 Custom Model Training and Evaluation

2.1 Training setup

Both models were trained for 25 epochs using the Adam optimizer (learning rate 10^{-3}), a batch size of 64, sparse categorical cross-entropy loss and a fixed random seed (42). They used the same 70/15/15 train/validation/test split and the same machine. Inputs are 64×64 RGB images scaled to $[0, 1]$.

2.2 Training and validation loss curves



(a) Model A **(b)** Model B

Figure 3: Training and validation loss over 25 epochs.

Model A shows strong overfitting (Figure 3a). The training loss falls steadily to about 0.03, but the validation loss reaches its minimum of about 0.52 around epoch 4–5 and then rises to about 1.3 by epoch 25. Model A has about 1.14 M parameters and only about 12 k training images, so it memorises the training set instead of learning features that generalise.

Model B behaves very differently (Figure 3b). Training and validation loss decrease together from about 1.5 to about 0.70 and the gap between them stays small. Its small capacity (about 20 k parameters) acts as an implicit regulariser. The curves are still decreasing at epoch 25, so Model B is probably under-trained rather than over-fitted. Validation loss lying slightly below training loss early on is normal, because the training loss is averaged over the whole epoch while the model is still improving.

2.3 Test-set evaluation

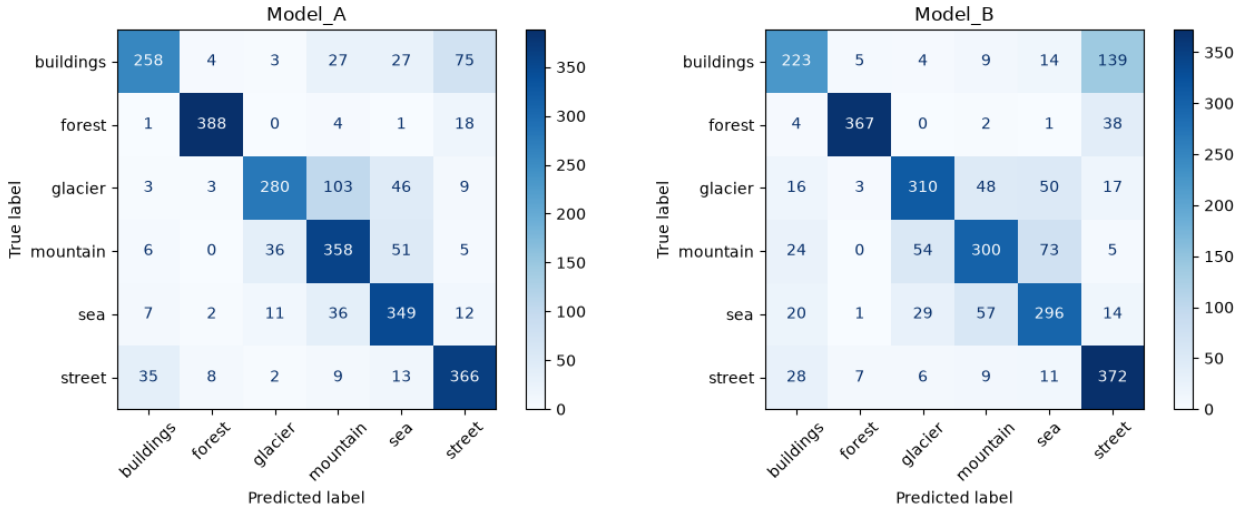
Table 3 gives the overall test metrics and Table 4 the per-class precision and recall. The test set contains 2556 images.

Table 3: Overall test-set performance (macro averages over the six classes).

Model	Accuracy	Precision	Recall	F1
Model A	78.21 %	0.795	0.782	0.783
Model B	73.08 %	0.740	0.730	0.731

Table 4: Per-class test precision (P) and recall (R).

	Model A		Model B	
Class	P	R	P	R
Buildings	0.832	0.655	0.708	0.566
Forest	0.958	0.942	0.958	0.891
Glacier	0.843	0.631	0.769	0.698
Mountain	0.667	0.785	0.706	0.658
Sea	0.717	0.837	0.665	0.710
Street	0.755	0.845	0.636	0.859



(a) Model A **(b)** Model B

Figure 4: Test-set confusion matrices (rows: true label, columns: predicted label).

Observations from the confusion matrices (Figure 4):

- **Forest** is the easiest class for both models (F1 of 0.95 for Model A and 0.92 for Model B) because its colour and texture are distinctive.
- **Model A** mainly confuses glacier with mountain: 103 of 444 glacier images (23 %) were labelled mountain and 46 more were labelled sea. These classes share snow, rock, sky and water textures, which explains glacier's high precision (0.84) but low recall (0.63).
- **Model B** mainly confuses buildings with street: 139 of 394 building images (35 %). It also over-predicts street, with 585 images labelled street against 433 true ones, giving a high recall

(0.86) but a low precision (0.64) for that class. Both classes contain man-made structures that its limited capacity cannot separate well.

- Mountain, sea and glacier are confused with each other in both models, but more in Model B (for example mountain → sea: 73 images, versus 51 for Model A).

2.4 Comparison of Model A and Model B

Table 5 compares the two custom networks on the quantities requested in the assignment. Parameter counts come from the Keras summaries (Tables 1 and 2), the file size is that of the saved `.keras` checkpoint, and the training time is the mean wall-clock time per epoch over the 25 training epochs. The multiply–accumulate (MAC) counts were calculated analytically from the layer shapes of the convolutional and dense layers.

Table 5: Comparison of Model A (standard CNN) and Model B (depthwise separable CNN).

Metric	Model A	Model B	A : B
Total trainable parameters	1,142,726	20,097	56.9×
Float32 weights only (KB)	4,463.8	78.5	56.9×
Saved `.keras` file (KB)	13,435.8	284.1	47.3×
MACs per image (analytic)	42.3 M	5.15 M	8.2×
Training time per epoch (s)	12.74	7.30	1.75×
Test accuracy (%)	78.21	73.08	−5.13 pp
Macro F1-score	0.783	0.731	−0.052

The saved checkpoint is about three times larger than the raw float32 weights because Keras also stores the Adam optimizer state (two moment estimates for every weight) in the file. A model exported for inference only would therefore be correspondingly smaller. TensorFlow reported that GPU support was unavailable on the Windows machine used for training, so both models were trained on the CPU.

2.5 Trade-offs of depthwise separable convolutions

Parameters and storage. Replacing the three standard convolutions by depthwise separable ones reduces the convolutional parameters from 93,248 to 11,451 (8.1×). This is only a small part of the overall 56.9× reduction. In Model A the flatten layer feeds 8,192 features into a 128-unit dense layer, and this single layer holds 1,048,704 of the 1,142,726 parameters (91.8 %). Model B uses global average pooling instead, so its whole dense head has only 8,646 parameters. The comparison therefore reflects two design changes together, separable convolution and global average pooling, and an ablation would be needed to separate their individual effects on accuracy.

Computation. Factorising a $k \times k$ convolution into a depthwise and a pointwise stage reduces the cost per output position from $k^2 C_{\text{in}} C_{\text{out}}$ to $C_{\text{in}}(k^2 + C_{\text{out}})$ MACs. The cost ratio is $1/C_{\text{out}} + 1/k^2$, which is a 7.0×, 7.9× and 8.4× reduction for the three layers of Model B ($C_{\text{out}} = 32, 64, 128$, $k = 3$) and 8.0× over the whole convolutional stack (41.3 M versus 5.14 M MACs). Including the dense layers, Model A needs 42.3 M MACs per image and Model B 5.15 M. The profiler estimate of 10.77 MFLOPs for Model B reported in Section 4 agrees with this count, since one MAC corresponds to two floating-point operations.

Training time. The 8× reduction in MACs gives only a 1.75× reduction in training time per epoch (7.30 s versus 12.74 s). Depthwise convolutions perform few operations per byte of memory moved and are usually less well optimised in deep-learning libraries than dense convolutions, while the input pipeline, pooling and activation layers cost the same in both models. These factors probably explain why a reduction in MACs does not translate proportionally into wall-clock speed-up.

Accuracy. Model B loses 5.13 percentage points of test accuracy and 0.052 of macro F1. Its capacity is much smaller (32–128 filters per layer and a 64-unit hidden layer), and Figure 3b shows that it had not yet converged after 25 epochs. Model A, in contrast, was evaluated after the last epoch with no early stopping, when its validation loss had risen from about 0.52 to 1.3, so its reported accuracy may understate what it could reach with early stopping. The gap between the two models could therefore change with longer training for Model B or early stopping for Model A.

Summary. Depthwise separable convolutions together with global average pooling give a 57× smaller model, an 8× cheaper network in MACs and 1.75× faster training epochs, at the price of about 5 percentage points of accuracy. Whether this price is acceptable depends on the target device: a network with 78.5 KB of weights can run on hardware where a 4.4 MB network cannot.

3 Fine-Tuning Lightweight State-of-the-Art Networks

3.1 Choice of architectures

Two ImageNet-pretrained networks designed for mobile and edge computing were selected: MobileNetV2 [1] and EfficientNet-B0 [2]. MobileNetV2 is built from depthwise separable convolutions, the same operation used in Model B [3], arranged in inverted residual blocks with linear bottlenecks and ReLU6 activations. EfficientNet-B0 is the baseline of the EfficientNet family. It was found by architecture search, uses mobile inverted-bottleneck blocks with squeeze-and-excitation and the Swish activation, and its family scales depth, width and input resolution together. Together they allow Model B to be compared with a network built from the same basic operation (MobileNetV2) and with a stronger but larger design (EfficientNet-B0).

3.2 Transfer-learning procedure

Both networks were adapted with the same two-stage procedure:

- **Data.** The same stratified 70/15/15 splits as for the custom models (11,923 training, 2,555 validation and 2,556 test images) at 64×64 resolution, a batch size of 32 and seed 42.
- **Backbone.** ImageNet weights without the original classifier, with a $64 \times 64 \times 3$ input. The pretrained filters were learned on 224×224 images, so this is a deliberate low-resolution setting, and the backbone output is a $2 \times 2 \times 1280$ feature map.
- **Preprocessing.** MobileNetV2 uses its own `preprocess_input` (pixels scaled to $[-1, 1]$) inside the model. EfficientNet-B0 receives raw pixels in $[0, 255]$ because its rescaling is part of the Keras model. The custom models receive pixels scaled to $[0, 1]$.
- **New head.** Global average pooling, a 128-unit ReLU dense layer, dropout (0.3) and a 6-unit Softmax layer, which has 164,742 trainable parameters. The loss is sparse categorical cross-entropy.
- **Stage 1 (head training).** The backbone is frozen (BatchNorm in inference mode) and the head is trained with Adam at a learning rate of 10^{-3} for at most 5 epochs, with early stopping on the validation loss (patience 3, best weights restored).
- **Stage 2 (fine-tuning).** The last 20 % of the backbone layers are unfrozen, with all BatchNorm layers kept frozen, and training continues with Adam at 10^{-5} for at most 15 epochs, with early stopping on the validation loss (patience 5, best weights restored).

Training the head first prevents the randomly initialised layers from sending large, noisy gradients into the pretrained features. The much smaller fine-tuning learning rate lets the high-level features adapt to scene images without destroying the generic low-level filters, which transfer well between tasks [4], and freezing BatchNorm preserves the pretrained normalisation statistics. Table 6 summarises the validation behaviour of the two stages.

Table 6: Validation behaviour of the two fine-tuning stages (weights from the best epoch were restored).

Model	Stage	Trainable params	Epochs run	Best epoch	Val. loss	Val. acc. (%)
MobileNetV2	1 (head)	164,742	5	2	0.4333	84.31
	2 (fine-tune)	1,767,622	8	3	0.3929	86.26
EfficientNet-B0	1 (head)	164,742	5	3	0.3614	86.81
	2 (fine-tune)	2,452,246	15	10	0.3425	88.30

3.3 Test-set results

Table 7 reports the test performance, parameter count and model size of the two fine-tuned networks, and Table 8 gives the per-class precision and recall. The model size is given both as the size of the float32 weights alone and as the size of the saved `.keras` checkpoint (in MiB), which also contains the Adam optimizer state of the trainable weights.

Table 7: Test-set performance and size of the fine-tuned networks (macro averages over the six classes).

Model	Total params	Weights (MB)	Saved (MB)	Accuracy (%)	Precision	Recall	F1
MobileNetV2	2,422,726	9.24	23.30	85.92	0.862	0.861	0.861
EfficientNet-B0	4,214,313	16.08	35.61	88.30	0.885	0.885	0.885

Table 8: Per-class test precision (P) and recall (R) of the fine-tuned networks.

	MobileNetV2		EfficientNet-B0	
Class	P	R	P	R
Buildings	0.866	0.868	0.893	0.886
Forest	0.983	0.968	0.981	0.983
Glacier	0.836	0.770	0.815	0.802
Mountain	0.771	0.833	0.824	0.820
Sea	0.827	0.847	0.898	0.931
Street	0.892	0.878	0.897	0.889

3.4 Discussion

Accuracy. Both pretrained networks clearly outperform the custom models. MobileNetV2 reaches 85.92 %, which is 7.71 points above Model A and 12.84 points above Model B, and EfficientNet-B0 reaches 88.30 %, which is 10.09 and 15.22 points above them. The training set contains only about 12,000 images, so features learned on ImageNet are much more useful than features learned from scratch. The pretrained networks were also useful after a single epoch: the validation accuracy after the first epoch of head training was already 82.8 % (MobileNetV2) and 85.9 % (EfficientNet-B0), whereas Model B reached only 74.0 % after 25 epochs.

Effect of fine-tuning. Unfreezing the last 20 % of the backbone improved the validation accuracy at the selected checkpoint from 84.31 % to 86.26 % for MobileNetV2 (+1.95 points) and from 86.81 % to 88.30 % for EfficientNet-B0 (+1.49 points). The gain is modest compared with the

gain from transfer learning itself, which shows that most of the benefit comes from the pretrained features and the new head. Early stopping ended the MobileNetV2 fine-tuning after 8 epochs, with the best validation loss at epoch 3, and EfficientNet-B0 kept improving slowly until epoch 10.

Per-class behaviour. Forest is the easiest class for both networks (F1 of 0.976 and 0.982), as for the custom models. Glacier and mountain remain the hardest classes (F1 between 0.80 and 0.82), which agrees with the confusion pattern of the custom models, where these classes share snow, rock, sky and water textures. EfficientNet-B0 is better than MobileNetV2 in F1 for all six classes, with the largest improvement for sea (0.837 to 0.914).

Reliability of the comparison. Each model was trained once with a fixed seed. With 2,556 test images, the 95 % binomial confidence interval of an accuracy near 86–88 % is roughly ± 1.3 points. The 2.38-point difference between EfficientNet-B0 and MobileNetV2 is therefore modest relative to this uncertainty, whereas the differences of 12.8 points or more with respect to Model B are clearly significant.

4 Final Comparison and Trade-off Discussion

4.1 Benchmarking method

Model B was compared with the two fine-tuned networks in terms of accuracy, memory footprint and computational cost. The test accuracy was measured on the same 2,556 test images for all models. The saved Model B was reloaded before benchmarking and again gave 73.08 %, so the reloaded network behaves identically to the original. The computational cost is the number of floating-point operations (FLOPs) of one forward pass for a $1 \times 64 \times 64 \times 3$ input, estimated with the TensorFlow profiler. Inference speed was measured on an NVIDIA T4 GPU (Google Colab) with a batch of 32 random 64×64 images, 20 warm-up runs and 100 timed runs. The median batch latency, the time per image (batch latency divided by 32) and the throughput (32 divided by the mean batch time) are reported, and each timed run copied the result back to the host so that the GPU work was complete. The scaling of Model B's input to $[0, 1]$ was applied before timing, whereas the preprocessing of the pretrained networks is part of their graphs, but this is a single element-wise operation and its effect is negligible.

4.2 Results

Table 9 summarises all metrics, and Figures 5 and 6 visualise the trade-off. Relative to Model B, MobileNetV2 has 121× and EfficientNet-B0 210× more parameters, their saved checkpoints are 84× and 128× larger, they need 4.7× and 6.2× more FLOPs, and their batch latency is 4.5× and 2.9× higher.

Table 9: Accuracy, memory and computational cost of Model B and the fine-tuned networks (best value in each row in bold; GPU timings: NVIDIA T4, batch size 32).

Metric	Model B	MobileNetV2	EfficientNet-B0
Test accuracy (%)	73.08	85.92	**88.30**
Macro F1-score	0.731	0.861	**0.885**
Total parameters	**20,097**	2,422,726	4,214,313
Float32 weights (MB)	**0.077**	9.24	16.08
Saved model size (MiB)	**0.277**	23.302	35.612
Estimated FLOPs (million / image)	**10.765**	50.466	66.977
Median batch latency (ms)	**3.091**	13.969	8.924
Time per image (ms)	**0.097**	0.437	0.279
Throughput (images/s)	**9,365**	2,063	3,503

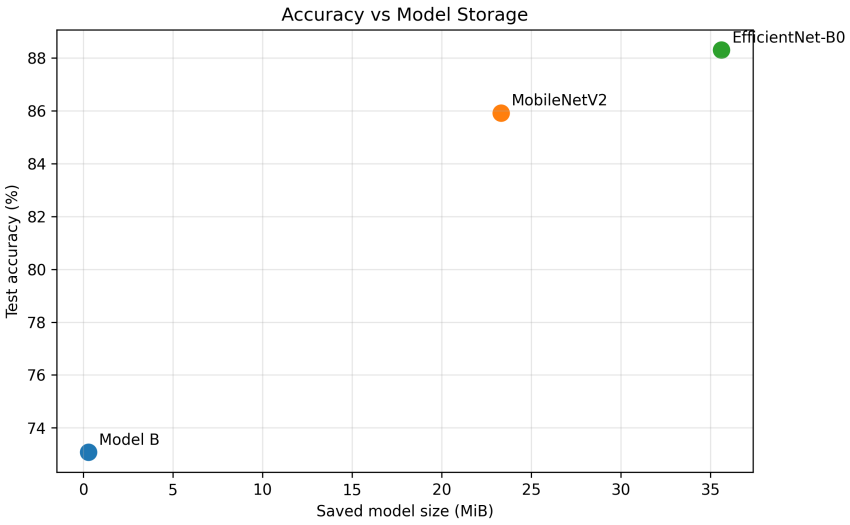
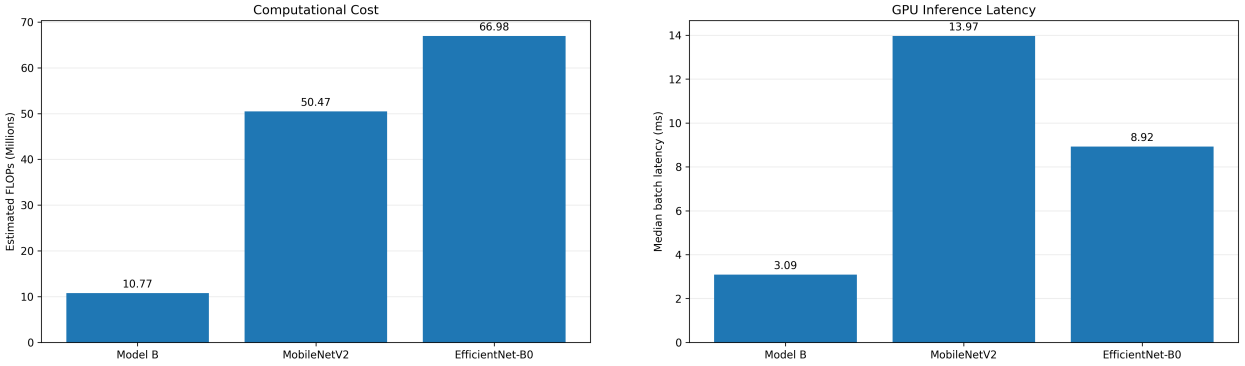


Figure 5: Test accuracy versus size of the saved model.



(a) Estimated FLOPs per image

(b) Median GPU batch latency

Figure 6: Computational cost and measured inference latency (batch size 32, NVIDIA T4).

Table 10 gives the per-class F1-scores of the three models.

Table 10: Per-class test F1-score of Model B and the fine-tuned networks.

Class	Model B	MobileNetV2	EfficientNet-B0
Buildings	0.629	0.867	0.889
Forest	0.923	0.976	0.982
Glacier	0.732	0.802	0.808
Mountain	0.681	0.801	0.822
Sea	0.687	0.837	0.914
Street	0.731	0.885	0.893

4.3 Accuracy

Transfer learning reduces the test error from 26.9 % for Model B to 14.1 % for MobileNetV2 and 11.7 % for EfficientNet-B0. Every class improves (Table 10). The largest gains are for buildings (F1 from 0.629 to 0.867 and 0.889) and sea, the classes on which the small custom network was weakest, while forest is already well separated by all three models. Model B had no batch normalisation, no data augmentation and a fixed learning rate, and its curves were still decreasing at epoch 25, so part of the accuracy gap could be reduced by further tuning. Relative to the cost, however, Model B retains 83 % of the accuracy of EfficientNet-B0 with only 0.5 % of its parameters and 16 % of its FLOPs. MobileNetV2 retains 97 % of the accuracy of EfficientNet-B0 with 57 % of its parameters and 75 % of its FLOPs.

4.4 Memory footprint

Model B needs only 78.5 KB for its float32 weights, compared with 9.24 MB for MobileNetV2 and 16.08 MB for EfficientNet-B0. Model B could be stored in the on-chip flash of many low-end microcontrollers, and 8-bit quantisation (not tested here) would reduce the weights to roughly a quarter of this size. The pretrained networks fit comfortably on a Raspberry Pi or a smartphone but would not fit on the smallest devices. The saved checkpoint sizes in Table 9 are larger than the weights because they include the optimizer state, so they overestimate the size of a deployed model and are not a measurement of runtime RAM.

Parameter count is also not the only memory requirement. The output of the first layer of Model B has $64 \times 64 \times 32 = 131{,}072$ activations, which is about 512 KB in float32 (128 KB at 8 bits) and is larger than the complete weight set. On a microcontroller the peak activation memory, which was not measured here, may therefore limit the model before the weight storage does.

4.5 Computational cost

The FLOPs ratios (4.7× and 6.2×) are much smaller than the parameter ratios (121× and 210×). At 64×64 input the spatial resolution is small, and the pretrained backbones keep most of their parameters in deep layers with many channels but tiny feature maps (the final map is only 2×2), where each weight is used at very few positions. The estimates are also consistent with the published cost of these networks at 224×224, about 300 million multiply–adds for MobileNetV2 [1] and about 390 million for EfficientNet-B0 [2]: scaling by the pixel-count ratio $(64/224)^2 \approx 0.082$ and counting two FLOPs per multiply–add gives roughly 49 and 64 MFLOPs.

Model B is also the fastest model on the GPU (3.09 ms per batch, 9,365 images/s). Its advantage over EfficientNet-B0 (2.9×) is smaller than its FLOPs advantage (6.2×), probably because a network this small does not keep a T4 busy, so fixed per-call overheads take a larger share of its time. On a CPU or microcontroller, where arithmetic is the bottleneck, the benefit could be larger. EfficientNet-B0 ran faster than MobileNetV2 (8.92 ms against 13.97 ms) although it needs 1.33× more FLOPs, so FLOPs alone are not a reliable predictor of latency: kernel implementations, memory access and the number of layers also matter. The timings come from one GPU and one batch size; they are batch latencies and not single-image or embedded-device latencies.

4.6 Custom sub-100k model versus pretrained lightweight model

Custom Model B (built from scratch).

- *Advantages:* the architecture can be sized to a given memory and compute budget (20,097 parameters, far below the 100,000 limit) and matched to the target hardware; it has the small-

est weights (78.5 KB) and the lowest FLOPs and GPU latency; it uses only a few standard operators, which simplifies deployment on constrained runtimes; and it trains quickly (7.3 s per epoch on a CPU) without depending on external weights.

- *Limitations:* the accuracy is the lowest (73.08 %); it must learn all features from only about 12,000 images and so cannot match the representations learned from ImageNet; it needs careful design and tuning (no batch normalisation or augmentation was used here); and its weakest classes are buildings and street, which contain similar man-made structures.

Fine-tuned pretrained networks (MobileNetV2, EfficientNet-B0).

- *Advantages:* accuracy is 12.8 to 15.2 points higher; they converge within a few epochs; they need little design effort; and the architectures are well tested.

- *Limitations:* they have 121 to 210 times more parameters, 9 to 16 MB of float32 weights, 4.7 to 6.2 times more FLOPs and 2.9 to 4.5 times higher GPU latency; the architecture is fixed, so they cannot be shrunk to an arbitrary budget; and they were designed for 224×224 inputs, so at 64×64 the pretrained filters are not used under the conditions they were trained for.

Choosing between them. For devices with very small flash and RAM, a network such as Model B is the only one of the three that fits, and its accuracy loss is the price of that constraint. For devices with a few megabytes of memory, such as a Raspberry Pi or a phone, the pretrained networks give much higher accuracy. Among them, MobileNetV2 is preferable when parameters and FLOPs matter most (it needs 57 % of the parameters and 75 % of the FLOPs of EfficientNet-B0 for 2.4 points less accuracy), whereas EfficientNet-B0 gives the best accuracy and was also faster on the GPU used here. Intermediate options, such as a MobileNetV2 with a reduced width multiplier, quantisation or pruning, or distilling EfficientNet-B0 into a Model B-sized network, could give a better balance but were not evaluated.

5 Conclusion

Two custom CNNs, a standard network (Model A, 1.14 M parameters) and a depthwise separable network (Model B, 20,097 parameters), and two fine-tuned lightweight networks (MobileNetV2 and EfficientNet-B0) were compared on six-class scene classification at 64×64 resolution. Replacing standard convolutions and the flatten–dense head with depthwise separable convolutions and global average pooling made the network 57× smaller and 8× cheaper in MACs for a loss of 5.1 points of accuracy. Transfer learning then improved accuracy by a further 12.8 to 15.2 points over Model B, at the cost of 121 to 210 times more parameters and 4.7 to 6.2 times more FLOPs. EfficientNet-B0 gave the best accuracy (88.30 %), MobileNetV2 gave a better accuracy-per-cost balance among the pretrained models, and Model B gave the lowest memory and computation. The experiments also show that parameter count, FLOPs and measured latency are different quantities and do not rank the models identically.

The study has several limitations. Each model was trained once with a single seed, so differences of about 2 points should be interpreted with care. The custom models were trained on a CPU and the pretrained models on a GPU, so their training times are not comparable. Inference was measured on a data-centre GPU, not on an embedded device, and peak RAM and energy use were not measured. Finally, the accuracy of Model B could probably be improved with augmentation, batch normalisation and learning-rate scheduling.

Code Repository

The notebooks, saved models and result figures are available at `https://github.com/RavPraveen/EN3150_Assignment_03`.

References

[1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[2] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2019, pp. 6105–6114.

[3] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv:1704.04861*, 2017.

[4] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 3320–3328.